ASSIGNMENT 20.5

AI Assisted Coding
2303A510B7
Batch 14

Task 1 – Password Storage Vulnerability Check
Task:
Analyze an AI-generated signup script for insecure password storage.
Instructions:
- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:
- A secure registration script with hashed password storage.

C: > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > Q1Ha20.2.py > ...

```python
#write a python program to Check signup script for insecure password storage
#Check whether passwords are stored in plain text.
#Improve the script using hashing (bcrypt or hashlib).

import bcrypt
def register_user(username, password):
    # Hash the password using bcrypt
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

    # Store the username and hashed password (for demonstration, we will just print it)
    print(f"Username: {username}")
    print(f"Hashed Password: {hashed_password.decode('utf-8')}")
# Example usage
register_user("user1", "securepassword123")
```

OUTPUT :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/Q1Ha2
0.2.py
Username: user1
Hashed Password: $2b$12$4gLjJCYzSZUkBR3xIz06zOkID2N13Q9PkRWVTsTeo1.9y6bcOeozq
PS C:\Users\ADMIN>
```

Task 2 – Hardcoded Secrets Detection
Task:
Evaluate AI-generated code for hardcoded API keys or credentials.
Instructions:
- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:
- A secure script with secrets managed properly.

C: > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > Q2ha20.5.py > ...

```python
#Ask AI to generate a script that connects to an external API.
#Identify if API keys/passwords are written directly in code.
#Refactor using environment variables or config files.
import os
import requests
def get_api_data():
    # Load API key from environment variable
    api_key = os.getenv('API_KEY')
    if not api_key:
        raise ValueError("API_KEY environment variable not set")
    url = "https://api.example.com/data"
    headers = {"Authorization": f"Bearer {api_key}"}
    response = requests.get(url, headers=headers)
    if response.status_code == 200:
        return response.json()
    else:
        raise Exception(f"API request failed with status code {response.status_code}")
# Example usage
if __name__ == "__main__":
    try:
        data = get_api_data()
        print(data)
    except Exception as e:
        print(e)
```

OUTPUT :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
0.5.py
API_KEY environment variable not set
```

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

```
C: > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > Q3ha20.5.py > ...
#•  Prompt AI to generate a Flask file upload example.
#•  Check for vulnerabilities like uploading executable files.
#•  Add validation for file type, size, and secure filename handling.
from flask import Flask, request, redirect, url_for, flash
from werkzeug.utils import secure_filename
import os
app = Flask(__name__)
app.secret_key = "secretkey"
# Configure upload folder
UPLOAD_FOLDER = 'uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
# Limit file size (2MB)
app.config['MAX_CONTENT_LENGTH'] = 2 * 1024 * 1024
# Allowed extensions
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'pdf', 'txt'}
# Allowed MIME types
ALLOWED_MIME_TYPES = {
    "image/png",
    "image/jpeg",
    "image/gif",
```

OUTPUT :

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using os.system() or subprocess.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

```
C: > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > 4ha20.5.py > ...
#Ask AI to generate code using os.system() or subprocess.
#Identify command injection risks with user input.
#Fix using safe subprocess calls and input validation.
import subprocess
def execute_command(user_input):
    # Validate user input (allow only alphanumeric and limited special characters)
    if not user_input.isalnum():
        return "❌ Invalid input: Only alphanumeric characters are allowed."

    # Use subprocess to execute the command safely
    try:
        result = subprocess.run(['echo', user_input], check=True, stdout=subprocess.PIPE, stderr=subp
        return f"✅ Command output: {result.stdout.strip()}"
    except subprocess.CalledProcessError as e:
        return f"❌ Command execution failed: {e.stderr.strip()}"
# Example usage
if __name__ == "__main__":
    user_input = input("Enter a command argument (alphanumeric only): ")
    print(execute_command(user_input))
```

OUTPUT :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/4ha20
.5.py
Enter a command argument (alphanumeric only):
```

Task 5 – Secure Input Handling in Web Forms

Task:

Check AI-generated web form code for improper input validation.

Instructions:

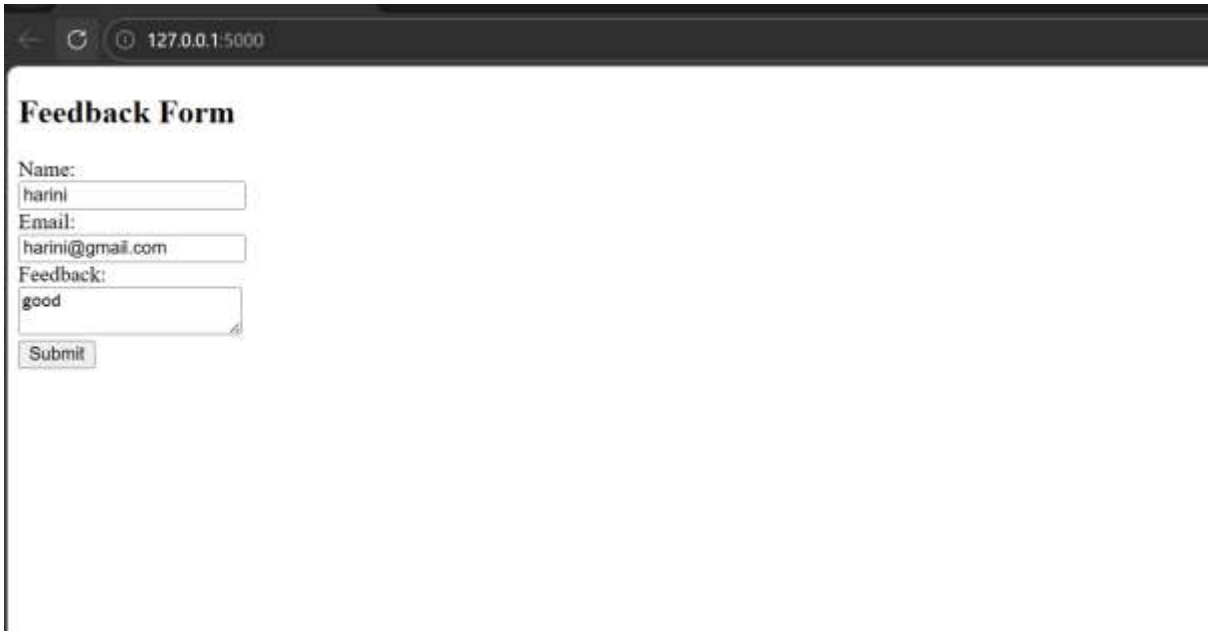
- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

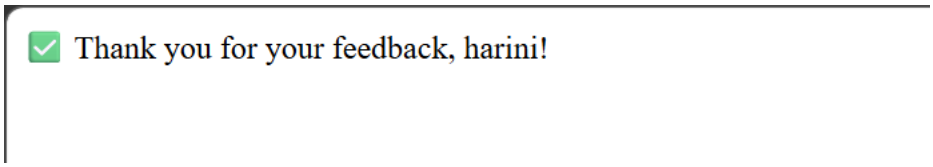
Expected Output:

A secure form resistant to injection attacks.

```
C: > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > Q5ha20.5.py > ...
#*  Ask AI to generate an HTML + Flask feedback form.
#*  Identify missing validation and unsafe rendering.
#*  Fix using server-side validation and sanitization.
from flask import Flask, request, render_template_string
import re
app = Flask(__name__)
# Simple HTML form template
form_template = '''
<!doctype html>
<title>Feedback Form</title>
<h2>Feedback Form</h2>
<form method=post>
  <label for="name">Name:</label><br>
  <input type=text name=name required><br>
  <label for="email">Email:</label><br>
  <input type=email name=email required><br>
  <label for="feedback">Feedback:</label><br>
  <textarea name=feedback required></textarea><br>
  <input type=submit value=Submit>
</form>
'''
def sanitize_input(user_input):
```

OUTPUT :


127.0.0.1:5000
Feedback Form
Name:
harini
Email:
harini@gmail.com
Feedback:
good
Submit


Thank you for your feedback, harini!